

Kiểm tra giữa kỳ Lập trình hướng đối tượng lớp INT2204 80

Thời gian làm bài 90 phút. Không được sử dụng tài liệu.

Câu 1 - Đúng/Sai (5 điểm)

1. (0,2 điểm)

Trình biên dịch Java biên dịch mã nguồn (.java) trực tiếp thành mã máy nhị phân của hệ điều hành.

2. (0,2 điểm)

Một lớp trong Java có thể implements nhiều interface nhưng chỉ có thể extends duy nhất một lớp cha.

3. (0,2 điểm)

Thuộc tính (field) khai báo với phạm vi private trong lớp cha sẽ không thể truy cập trực tiếp bằng tên trong mã nguồn của lớp con.

4. (0,2 điểm)

Phương thức khởi tạo (constructor) có thể được khai báo với phạm vi truy cập private.

5. (0,2 điểm)

Từ khóa this có thể được sử dụng bên trong phương thức tĩnh (static method) để tham chiếu tới đối tượng hiện tại.

6. (0,2 điểm)

Nếu người lập trình khai báo ít nhất một constructor có tham số cho lớp, trình biên dịch vẫn tự động bổ sung constructor không tham số mặc định.

7. (0,2 điểm)

Lời gọi constructor cha super(...) nếu có phải là câu lệnh đầu tiên bên trong constructor của lớp con.

8. (0,2 điểm)

Phương thức trừu tượng (abstract method) trong abstract class có thể khai báo là private hoặc static.

9. (0,2 điểm)

Tất cả các phương thức khai báo trong một interface (từ Java 8) bắt buộc phải là phương thức trừu tượng không có thân hàm.

10. (0,2 điểm)

Nạp chồng phương thức (overloading) xảy ra khi hai phương thức trong cùng một lớp có cùng tên nhưng khác nhau về danh sách tham số.

11. (0,2 điểm)

Phương thức được ghi đè (overriding) ở lớp con phải có phạm vi truy cập rộng hơn hoặc bằng phạm vi truy cập của nó ở lớp cha.

12. (0,2 điểm)

Không thể khởi tạo đối tượng trực tiếp (bằng từ khóa new) từ một interface hoặc một abstract class.

13. (0,2 điểm)

Trong Java, biến tham chiếu thuộc kiểu lớp cha có thể trở tới một đối tượng thuộc lớp con của nó (Upcasting).

14. (0,2 điểm)

Toán tử instanceof trả về true khi kiểm tra một tham chiếu đang giữ giá trị null với bất kỳ kiểu dữ liệu nào.

15. (0,2 điểm)

Từ khóa final khi đặt trước khai báo lớp sẽ ngăn không cho các lớp khác kế thừa lớp đó.

16. (0,2 điểm)

Từ khóa final đặt trước khai báo một thuộc tính biến đối tượng sẽ đảm bảo trạng thái bên trong của đối tượng đó hoàn toàn không bị thay đổi.

17. (0,2 điểm)

Phương thức static ở lớp con có thể ghi đè (override) một phương thức static có cùng ký hiệu ở lớp cha.

18. (0,2 điểm)

Một giao diện (interface) có thể kế thừa (extends) từ nhiều giao diện khác.

19. (0,2 điểm)

Ngoại lệ thuộc nhóm Unchecked Exception (kế thừa từ RuntimeException) bắt buộc phải khai báo trong mệnh đề throws của phương thức.

20. (0,2 điểm)

Khối finally trong cấu trúc try-catch-finally luôn được thực thi trừ khi chương trình bị dừng đột ngột (ví dụ System.exit()).

21. (0,2 điểm)

Hai đối tượng trong Java có phương thức equals() trả về true thì bắt buộc hai biến tham chiếu tới chúng phải trỏ cùng địa chỉ ô nhớ (so sánh ==).

22. (0,2 điểm)

Phương thức khởi tạo của lớp con sẽ tự động gọi super() không tham số của lớp cha nếu lớp con không gọi super(...) tường minh.

23. (0,2 điểm)

Câu lệnh package dùng để nhóm các lớp liên quan và phải nằm ở dòng mã đầu tiên (không tính comment/dòng trống) trong tệp Java.

24. (0,2 điểm)

Một lớp con có thể kế thừa và gọi phương thức static public của lớp cha thông qua tên của lớp con.

25. (0,2 điểm)

Trong Java, mảng (Array) là một kiểu dữ liệu tham chiếu và kích thước của mảng không thể thay đổi sau khi khởi tạo.

Câu 2 - Tự luận (1,5 điểm)

26. (0,75 điểm)

Phân biệt cơ chế Ghi đè phương thức (Method Overriding) và Nạp chồng phương thức (Method Overloading) trong Java. Cho ví dụ mã Java minh họa cho từng trường hợp.

27. (0,75 điểm)

Trình bày sự khác biệt giữa Lớp trừu tượng (Abstract Class) và Giao diện (Interface) trong Java (về tính đa kế thừa, thuộc tính, phương thức, và constructor). Khi nào nên sử dụng Lớp trừu tượng và khi nào nên dùng Interface?

Câu 3 - Một lựa chọn (2 điểm)

28. (0,4 điểm)

Cho đoạn mã Java sau:

```
class A { void show() { System.out.print("A "); } }  
class B extends A { void show() { System.out.print("B "); } }  
A obj = new B(); obj.show();
```

Kết quả in ra màn hình khi chạy đoạn mã trên là gì?

- a. A
- b. B
- c. Biên dịch lỗi
- d. Ngoại lệ ClassCastException

29. (0,4 điểm)

Phát biểu nào sau đây là ĐÚNG về phạm vi truy cập protected trong Java?

- a. Chỉ có thể được truy cập từ bên trong cùng một lớp.
- b. Chỉ được truy cập từ các lớp con, bất kể lớp con nằm ở package nào.
- c. Có thể được truy cập từ các lớp trong cùng package và các lớp con nằm ở package khác.
- d. Có phạm vi truy cập rộng hơn public.

30. (0,4 điểm)

Cho phương thức void process(int[] arr) trong Java. Nếu bên trong phương thức thực hiện arr[0] = 99;, điều gì xảy ra với mảng gốc được truyền vào từ nơi gọi?

- a. Mảng gốc không bị thay đổi vì Java truyền tham số theo giá trị.
- b. Mảng gốc bị thay đổi phần tử đầu tiên thành 99 vì arr là tham chiếu trỏ tới cùng đối tượng mảng trong Heap.
- c. Đoạn mã gây lỗi biên dịch.
- d. Tạo ra một bản sao mới của mảng và chỉ bản sao bị thay đổi.

31. (0,4 điểm)

Khối lệnh catch nào sau đây là ĐÚNG quy tắc khi xử lý ngoại lệ có quan hệ kế thừa trong Java?

- a. Khối catch (Exception e) phải đứng trước khối catch (NullPointerException e).
- b. Khối catch (NullPointerException e) phải đứng trước khối catch (Exception e).
- c. Thứ tự giữa Exception và NullPointerException không quan trọng, trình biên dịch tự động xử lý.
- d. Không được phép sử dụng nhiều khối catch cho cùng một khối try.

32. (0,4 điểm)

Phát biểu nào sau đây mô tả ĐÚNG NHẤT về từ khóa final khi đặt trước một phương thức của lớp?

- a. Phương thức đó không thể được gọi từ bên ngoài lớp.
- b. Phương thức đó bắt buộc phải được ghi đè ở các lớp con.
- c. Phương thức đó không thể bị ghi đè (override) bởi bất kỳ lớp con nào.
- d. Phương thức đó tự động trở thành phương thức tĩnh (static).

Câu 4 - Đọc và sửa mã Java (1,5 điểm)

```

abstract class Shape {
    private String color;
    public Shape(String color) {
        this.color = color;
    }
    public abstract double getArea();
    public void printInfo() {
        System.out.println("Color: " + color + ", Area: " + getArea());
    }
}

```

```

public class Rectangle extends Shape {
    private double width;
    private double height;

    public Rectangle(double w, double h) {
        width = w;
        height = h;
    }

    public Rectangle(String c, double w, double h) {
        width = w;
        height = h;
        super(c);
    }

    public double getArea() {
        return width * height;
    }
}

```

```

public class Circle extends Shape {
    private double radius;

    public Circle(String c, double r) {
        super(c);
        radius = r;
    }
}

```

```

public class TestShape {
    public static void main(String[] args) {
        Shape s1 = new Shape("Red");
        Shape s2 = new Rectangle("Blue", 4.0, 5.0);
        s2.printInfo();
    }
}

```

33. (1 điểm)

Chương trình Java trên có những lỗi biên dịch nào? Hãy chỉ ra các câu lệnh gây lỗi, giải thích nguyên nhân và đưa ra phương án sửa để chương trình biên dịch thành công.

34. (0,5 điểm)

Sau khi sửa các lỗi biên dịch ở câu 4.a (với Rectangle s2 giữ nguyên màu "Blue", width 4.0, height 5.0), câu lệnh `s2.printInfo();` trong hàm main sẽ in ra màn hình kết quả chính xác như thế nào? Giải thích cơ chế gọi phương thức.

ĐÁP ÁN VÀ HƯỚNG DẪN CHẤM

Câu 1 - Đúng/Sai (5 điểm)

1. Sai
2. Đúng
3. Đúng
4. Đúng
5. Sai
6. Sai
7. Đúng
8. Sai
9. Sai
10. Đúng
11. Đúng
12. Đúng
13. Đúng
14. Sai
15. Đúng
16. Sai
17. Sai
18. Đúng
19. Sai
20. Đúng
21. Sai
22. Đúng
23. Đúng
24. Đúng
25. Đúng

Câu 2 - Tự luận (1,5 điểm)

26. Overriding xảy ra giữa lớp cha - con (cùng tên, cùng tham số, cùng kiểu trả về, quyết định lúc runtime - dynamic binding). Overloading xảy ra trong cùng lớp hoặc lớp con (cùng tên, khác danh sách tham số, quyết định lúc compile-time - static binding). Cần có mã Java minh họa hợp lệ cho cả hai.

- Phân biệt đúng bản chất Overriding (quan hệ cha-con, dynamic binding) và Overloading (cùng tên khác tham số, static binding) (0,5 điểm)

- Có ví dụ mã Java minh họa đúng và rõ ràng cho cả Overriding và Overloading (0,25 điểm)

Hướng dẫn chấm: Kiểm tra định nghĩa Overriding vs Overloading, tính chất liên kết động/tĩnh và ví dụ Java minh họa.

27. Khác biệt: Abstract class hỗ trợ đơn kế thừa, chứa thuộc tính biến, constructor, phương thức có/ không có thân hàm. Interface hỗ trợ đa kế thừa (implements nhiều interface), thuộc tính mặc định public static final, không constructor, chứa phương thức abstract/default/static. Ngữ cảnh: Dùng Abstract Class khi các lớp có quan hệ họ hàng close is-a và chia sẻ mã/trạng thái chung. Dùng Interface khi định nghĩa hợp đồng hành vi (can-do) cho các lớp không nhất thiết có quan hệ kế thừa.

- So sánh đúng ít nhất 4 điểm khác biệt cơ bản giữa Abstract Class và Interface (kế thừa, field, constructor, method) (0,5 điểm)

- Nêu đúng ngữ cảnh nên dùng Abstract Class (quan hệ is-a, dùng chung trạng thái) và Interface (hợp đồng hành vi can-do) (0,25 điểm)

Hướng dẫn chấm: Chấm theo các tiêu chí so sánh (extends/implements, field, constructor, method) và lý do lựa chọn ngữ cảnh áp dụng.

Câu 3 - Một lựa chọn (2 điểm)

28. B. B

29. C. Có thể được truy cập từ các lớp trong cùng package và các lớp con nằm ở package khác.

30. B. Mảng gốc bị thay đổi phần tử đầu tiên thành 99 vì arr là tham chiếu trỏ tới cùng đối tượng mảng trong Heap.

31. B. Khối catch (NullPointerException e) phải đứng trước khối catch (Exception e).

32. C. Phương thức đó không thể bị ghi đè (override) bởi bất kỳ lớp con nào.

Câu 4 - Đọc và sửa mã Java (1,5 điểm)

33. Các lỗi biên dịch:

1. Rectangle(double, double) ngầm gọi super() nhưng Shape không có constructor không tham số.

2. Trong Rectangle(String, double, double), super(c) phải là câu lệnh đầu tiên trong constructor.

3. Từ khóa Public viết hoa P trong getArea() là sai cú pháp (phải là public).

4. Lớp Circle kế thừa Shape nhưng không override phương thức trừu tượng getArea().

5. Trong main: Shape s1 = new Shape("Red"); vi phạm không được khởi tạo trực tiếp đối tượng từ abstract class.

- Phát hiện lỗi thiếu super() mặc định trong Rectangle(w,h) và super(c) không đứng ở vị trí đầu tiên (0,25 điểm)

- Phát hiện lỗi cú pháp từ khóa Public viết hoa ở phương thức getArea() trong Rectangle (0,25 điểm)

- Phát hiện lỗi lớp Circle kế thừa Shape nhưng chưa override phương thức trừu tượng getArea() (0,25 điểm)

- Phát hiện lỗi khởi tạo đối tượng trực tiếp từ abstract class Shape trong hàm main (0,25 điểm)

Hướng dẫn chấm: Đối chiếu tính chính xác khi chỉ ra lỗi biên dịch, giải thích nguyên nhân và phương án sửa hợp lệ.

34. Kết quả in ra màn hình: Color: Blue, Area: 20.0

Giải thích: s2 có kiểu tham chiếu Shape nhưng trỏ tới đối tượng Rectangle. Khi s2.printInfo() chạy, phương thức printInfo() gọi getArea(), cơ chế liên kết động (dynamic binding) sẽ gọi getArea() của Rectangle, trả về $4.0 * 5.0 = 20.0$.

- Xác định đúng chuỗi kết quả in ra màn hình: Color: Blue, Area: 20.0 (0,25 điểm)

- Giải thích đúng cơ chế liên kết động (dynamic binding / polymorphism) khi gọi getArea() (0,25 điểm)

Hướng dẫn chấm: Chấm điểm dựa trên kết quả chuỗi in ra chính xác và giải thích đúng cơ chế liên kết động (dynamic binding).